NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

Centre for Professional and Continuing Education

(SCTP) Advanced Professional Certificate

Data Science and AI



L07
Neural
Networks

Sarah's day of building her first neural network

Video intro 1: https://www.youtube.com/watch?v=Xrhz4N-crO8
Video intro 2: https://youtu.be/vvF_0BKDS5Q
Video intro 3: gradient descent https://youtu.be/v6Y9BWn1oXg

Interactive html: https://su-ntu-ctp.github.io/6m-data-3.7-Neural-Networks-Deep-Learning/

Interactive visualisation of MLPs: https://playground.tensorflow.org/ try this to find out how to train a MLP with the combination of different hidden layers, neurons, input features, activation functions etc to get the desired output.

https://projector.tensorflow.org/

Copyright © 2026 Nanyang Technological University. All rights reserved.

Text

Join at

slido.com

#DS6307



Copyright © 2026 Nanyang Technological University. All rights reserved.

Where We Left Off

From time series to sequential behaviour

Marcus, end of L06:

"If we had DETAILED CUSTOMER BEHAVIOUR data — every click, every page view, every cart event —
could you predict whether they'd complete checkout WHILE they're shopping?"

What's different today

🧠 **Neural networks for the first time**	MLP with hidden layers + non-linear activations
🔥 **PyTorch as the framework**	The 5-line training loop you'll use forever
🎯 **Goal is INTUITION, not engineering**	L08-L10 are where deep learning beats classical
📊 **Same compare-to-baselines mindset**	MLP vs LR vs gradient boosting on the same data

Copyright © 2026 Nanyang Technological University. All rights reserved.

Why Do We Need Neural Networks?

Where linear & logistic regression, k-means, random forest and boosting hit their limits

Problem	Why classical models struggle	What neural networks do instead
Images	Pixels are not meaningful columns, so a person must hand-craft features first	Learn their own features: edges, then shapes, then objects
Text & language	Word counts ignore order: "not good" looks almost like "good"	Read each word in the context of the words around it
Audio & sequences	Each time step is a separate column; patterns across time need manual lag features	Learn patterns that unfold over time, such as speech or sensor streams
Very large datasets	Accuracy levels off once the model's simple shape is the limit	Keep improving as data and network size grow
Creating new content	Can only predict a number, a class or a cluster	Generate text, images and audio (the basis of GenAI)

Rule of thumb: spreadsheet data → try gradient boosting first. Images, text, audio → neural networks.

Copyright © 2026 Nanyang Technological University. All rights reserved.

What IS a Neural Network?

Three things, then we're done with the mystique

1

A Activation Function

A flexible, non-linear function from inputs to outputs.
Learned automatically — you specify the architecture, not the function.

▸ **Forward propagation: input → output**

2

A Loss Function

A measure of how wrong the function is on training data.
For classification: cross-entropy. For regression: MSE.

▸ **Loss tells gradient descent which way to walk**

3

A Learning Algorithm

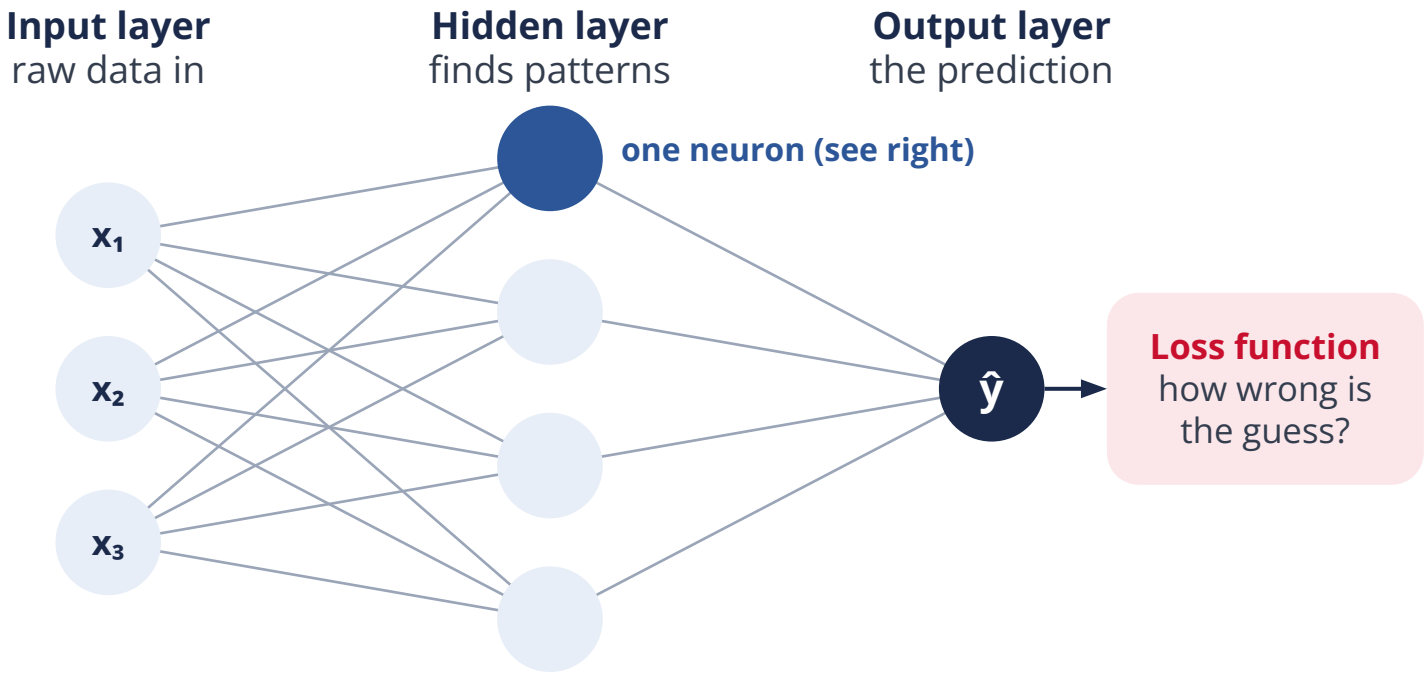
Gradient descent + back propagation.
Walk downhill on the loss surface, one small step at a time.

▸ **Updates: weights -= lr × gradient_of_loss**

Copyright © 2026 Nanyang Technological University. All rights reserved.

How a Neural Network Works: The Big Picture

Layers of simple units pass numbers forward to make a prediction, then learn from their mistakes



Forward propagation: data flows forward, activating each neuron to make a prediction

Back propagation: error flows back, nudging each weight to cut the loss with gradient descent

Inside one neuron: the perceptron

1. **Weigh:** multiply each input by a weight (how much it matters)
2. **Sum:** add them up, plus a bias
3. **Activate:** an activation function decides how strongly to pass the signal on (ReLU turns negatives into 0)

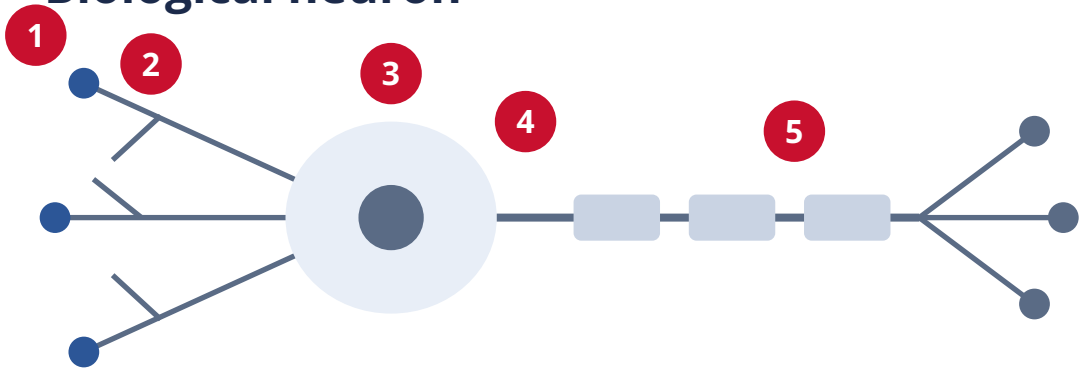
Without activation functions, stacking layers still only draws a straight line, like linear regression.

Copyright © 2026 Nanyang Technological University. All rights reserved.

The Perceptron: A Brain Cell Written in Maths

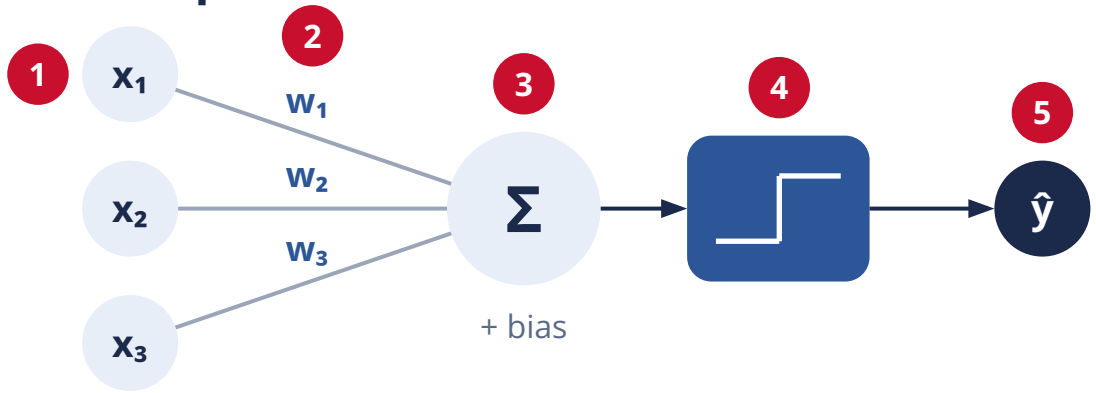
Like a neuron, a perceptron weighs its inputs, adds them up, then decides whether to fire

Biological neuron



signals flow left to right →

Perceptron

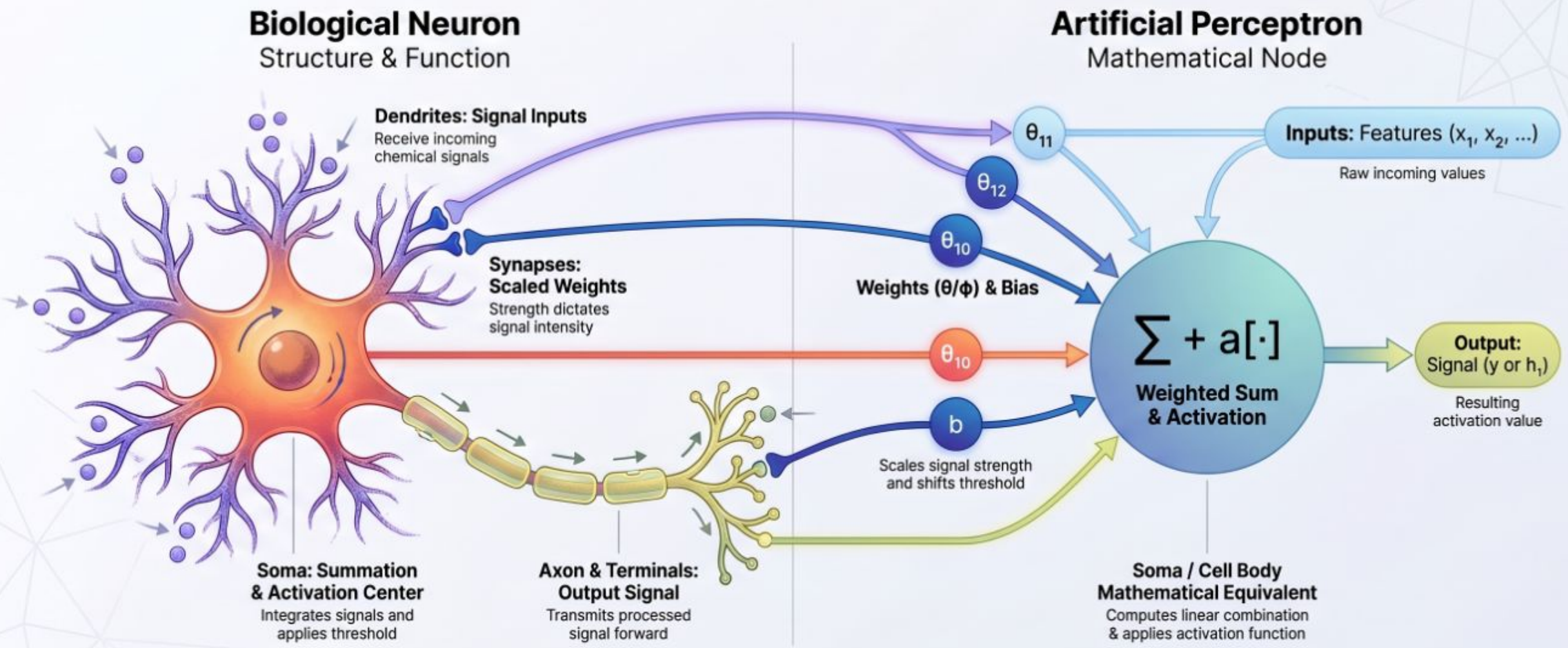


numbers flow left to right →

	Biological neuron	Perceptron
1	**Dendrites:** receive signals from other cells	**Inputs:** the features, e.g. x_1 = pages viewed
2	**Synapses:** some connections are stronger than others	**Weights:** how much each input matters
3	**Cell body (Soma):** adds up all incoming signals	**Weighted sum:** Σ (input × weight) + bias
4	**Threshold:** fires only if the total is high enough	**Activation function:** decides whether to fire
5	**Axon:** passes the signal to the next neuron	**Output ŷ:** goes to the next layer, or is the prediction

Copyright © 2026 Nanyang Technological University. All rights reserved.

Biological Neuron vs. **Artificial Perceptron:** Mapping Nature to Math



Artificial neural networks draw conceptual inspiration from biological nervous systems. By mapping organic components to their mathematical equivalents, we can understand how computational models simulate biological intelligence. This mapping includes **Input Integration**, **Non-Linear Activation Firing**, and **Forward Propagation**.

Copyright © 2026 Nanyang Technological University. All rights reserved.

The Perceptron — One Neuron

The 1958 building block

Formula

$$\text{output} = \text{activation}(w_1 \cdot x_1 + w_2 \cdot x_2 + \ldots + w_n \cdot x_n + b)$$

Activation function determines the model type

- **Linear (no activation)** = Linear regression
- **Threshold (step function)** = 1958 perceptron, binary output (+1 / -1) $f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 \text{ (or } 0) & \text{if } z < 0 \end{cases}$
- **Sigmoid** = Logistic regression (binary classification) $\sigma(x) = \frac{1}{1 + e^{-x}}$
- **Softmax** = Multinomial logistic regression (multiclass) $\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$

⚠ A single perceptron only learns LINEARLY SEPARABLE patterns, Fails on XOR.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Neural Networks & The Universal Approximation Theorem

From Multilayer Perceptron Structure to Continuous Function Representation.

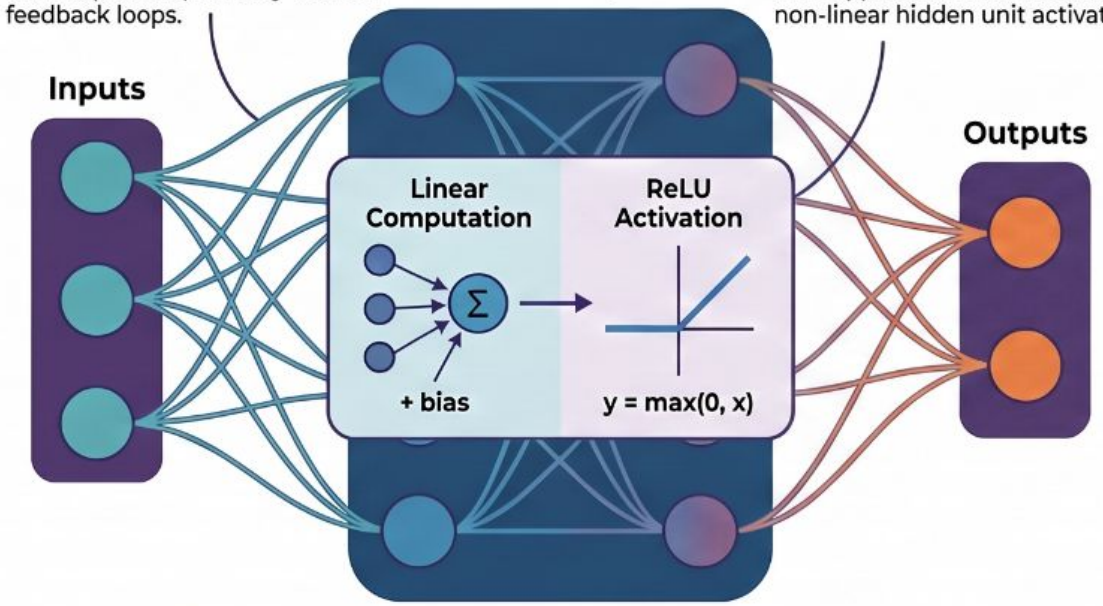
Multilayer Perceptron (MLP) Architecture

Layered Feedforward Structure

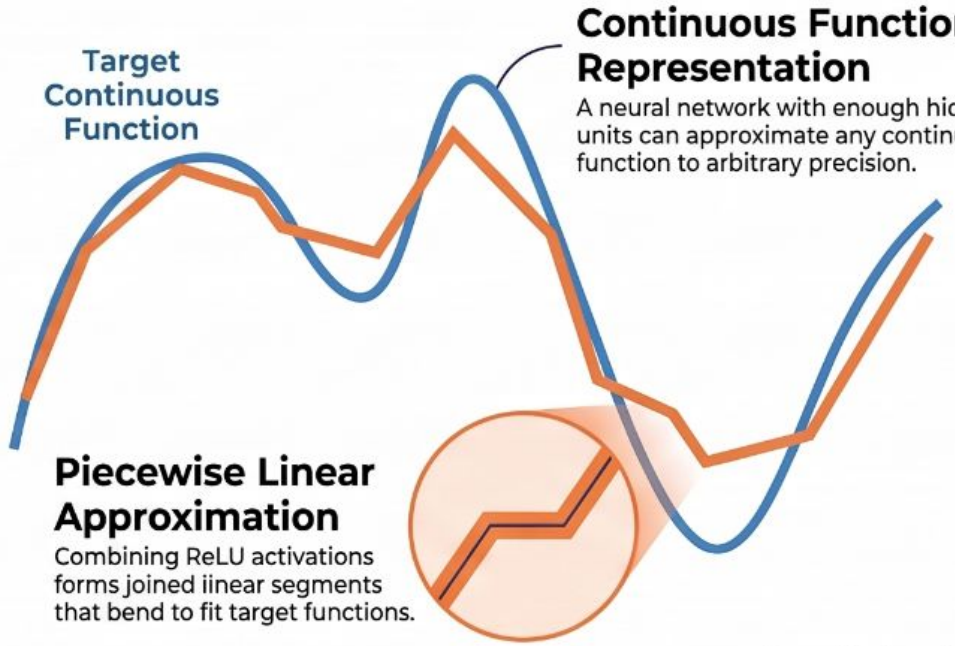
Connects inputs, hidden layers, and outputs sequentially without feedback loops.

Linear Computation and ReLU Activation

Computes linear transformations then applies ReLU functions to create non-linear hidden unit activations.



The Universal Approximation Theorem



Continuous Function Representation

A neural network with enough hidden units can approximate any continuous function to arbitrary precision.

Piecewise Linear Approximation

Combining ReLU activations forms joined iinear segments that bend to fit target functions.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Multi-Layer Perceptron (MLP)

Stack perceptrons + non-linear activations between layers

An MLP architecture

Input (9 features) → Linear → ReLU → Linear → ReLU → Linear → output

Each 'Linear' layer computes the weighted sum (weight × input + bias) for a group of neurons.

The 'ReLU' is the activation function applied immediately after, which provides the non-linearity that makes the network EXPRESSIVE.

The Universal Approximation Theorem

A single hidden layer with enough neurons + non-linear activations
can approximate ANY continuous function arbitrarily well.

On Sarah's data: 1 to 4 hidden layers (80 → ~12k weights) all hit test AUC ≈ 0.76.
For tabular data, start modest: 1-3 hidden layers, 16-128 neurons each.

Copyright © 2026 Nanyang Technological University. All rights reserved.

🟢 CODE-ALONG

Open: 02_perceptron_to_mlp.ipynb

What you'll do

1. Watch one perceptron fail on XOR (~50%) and an MLP solve it (~100%).
2. Train sklearn MLPClassifier (32 → 16) vs the logistic-regression baseline.
3. Sweep 5 architectures (1 to 4 hidden layers): does bigger help?

⏱️ **30 minutes · Stop at the 🟡 Extension marker**

Gradient Descent — The Mountain Analogy

Walk downhill on the loss surface

The mountain analogy

"You're blindfolded on a mountain. You can't see the valley, but you can feel the slope under your feet.
At each step, walk in the steepest downhill direction. Repeat until the ground feels flat."

The mapping to neural networks

Mountain (terrain)	→	`Loss function (function of all parameters)`
Your position	→	`Current values of all the model's weights`
Slope under your feet	→	`Gradient (partial derivatives of loss w.r.t. each weight)`
Step size	→	`Learning rate (lr) – the most important hyperparameter`
Repeat until ground is flat	→	`Train until gradient ≈ 0 (or you run out of patience)`

Copyright © 2026 Nanyang Technological University. All rights reserved.

Backpropagation — The Algorithm Behind Gradients

The chain rule, walking backwards through the network

What backprop computes

"For every weight in the network, what's the partial derivative of the loss with respect to that weight?" — applied via the chain rule from the output back to the inputs.

```
# Forward pass
predictions = model(inputs)
loss = criterion(predictions, targets)

# Backward pass — ONE LINE.
loss.backward()    # ← PyTorch computes all gradients via backprop

# Update
optimizer.step()
```

You don't derive backprop by hand. PyTorch (and TensorFlow, JAX, etc.) do it automatically via AUTOGRAD.

Copyright © 2026 Nanyang Technological University. All rights reserved.

🟢 CODE-ALONG

Open: 03_gradient_descent.ipynb

What you'll do

1. Walk downhill on $(w - 3)^2$ by hand, then let autograd find the gradient.
2. Swap the manual update for optim.SGD and build an nn.Module model.
3. Train your first PyTorch MLP (full-batch SGD, 200 epochs) on session data.

⏱️ **30 minutes · Stop at the 🟡 Extension marker**

PyTorch — Four Objects

Tensor · nn.Module · DataLoader · Optimizer

`Tensor`	**Multi-dim array with autograd** Replaces NumPy during training; tracks gradients automatically
`nn.Module`	**The base class for any model** You subclass it, define layers + forward()
`DataLoader`	**Wraps data for batched iteration** Used with TensorDataset for the typical workflow
`Optimizer`	**Updates parameters from gradients** torch.optim.Adam, SGD, etc. — separate from the model

Copyright © 2026 Nanyang Technological University. All rights reserved.

THE Training Loop

Memorise these 5 lines — used everywhere in deep learning

```
for epoch in range(n_epochs):
    for X_batch, y_batch in train_loader:        # iterate over mini-batches
        optimizer.zero_grad()                     # 1. clear old gradients
        predictions = model(X_batch)              # 2. forward pass
        loss = criterion(predictions, y_batch)    # 3. compute loss
        loss.backward()                           # 4. backward pass (autograd)
        optimizer.step()                          # 5. update parameters
```

The pattern transfers everywhere

L08 (CNNs on images): same 5 lines, different model.
L09 (NLP): same 5 lines, different model.
L10 (transformers): same 5 lines, different model.

Copyright © 2026 Nanyang Technological University. All rights reserved.

🟢 CODE-ALONG

Open: 04_pytorch_training_loop.ipynb

What you'll do

1. Build Dataset + DataLoader for mini-batched training.
2. Train with Adam optimiser + early stopping.
3. Compare PyTorch MLP to logistic regression, gradient boosting, sklearn MLP.

⏱️ **30 minutes · Stop at the 🟡 Extension marker**

End of Day — Sarah's Recommendation

What lands on Marcus's desk

Logistic Regression AUC 0.761	Tied top score, cheapest, most interpretable → SHIP THIS
Gradient Boosting AUC 0.749	Surprisingly the lowest of the four here
PyTorch MLP AUC 0.756	Foundation for L08-L10; not the production choice here

Marcus: 'Excellent. Next — automatically tag product PHOTOS as dress, jeans, jacket. Can you?'

Copyright © 2026 Nanyang Technological University. All rights reserved.

Bridge to Lesson 8

Marcus, after the end-of-day presentation:

"Great. Now — we're launching an Instagram product feed.
Can you automatically tag product PHOTOS so they categorise as
'dress', 'jeans', 'jacket', 'shoes', 'accessories', etc.?"

Lesson 8 — Computer Vision

CNN architecture · transfer learning · data augmentation

Same PyTorch training loop. New model architecture. Real images.

Before L08 — do ONE L07 assignment track: loan default (Foundational) or housing (Advanced).

Three Things to Remember

Beyond the algorithms

1

Neural networks are FUNCTIONS, not magic

They map inputs to outputs through stacked weighted sums + non-linear activations. Training is gradient descent on the loss surface.

2

The PyTorch training loop is THE pattern

5 lines that work for every deep-learning model you'll meet: zero_grad, forward, loss, backward, step. Memorise it.

3

On tabular data, the algorithm rarely matters

Neural networks shine on data with rich hierarchical structure (images, text). On tabular, they're competitive but rarely transformative.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Join at

slido.com

#DS6307



Copyright © 2026 Nanyang Technological University. All rights reserved.